

EvalBracket: Testing the Test Before Trusting a Below-Threshold Result

Ratnaditya Jonnalagadda

<https://github.com/Ratnaditya-J/EvalBracket>

July 2026

Abstract

Capability evaluations increasingly inform deployment, security, and safeguard decisions for frontier models. A negative result, however, has two different explanations: the model may lack the capability, or the evaluation setup may have failed to expose capability that was present. We introduce EvalBracket, an access-conditioned reporting method that first qualifies an elicitation battery on models with known-hidden capability, then limits the conclusion that the battery may support on a target checkpoint. We test the method in a preregistered model-organism experiment using five open model families, five benign multiple-choice tasks, three random seeds, three concealment mechanisms, six prompt routines, and Low-Rank Adaptation (LoRA) at three budgets. At a 60% policy threshold, the frozen construction produced 62 retained-capability hidden positives and 22 genuine negatives. Prompt-only batteries recovered 3.6% to 48.1% of held-out controls, with one-sided 95% lower bounds of 0.6% to 36.0%, and failed the preregistered 80% qualification gate. LoRA batteries recovered 93.3% to 100%, with lower bounds of 82.1% to 94.4%, and passed. Consequently, API-level negative results became *Upper bound unknown*, while qualified weight-access evaluation reduced false-below conclusions to 1.6% to 4.8% and retained 95.5% to 100% of correct-below conclusions. The experiment validates the decision mechanism on harmless proxies. It does not establish the dangerous capability of any frontier model or validate the method on operational biological, cyber, chemical, or radiological evaluations.

Keywords: capability evaluation, elicitation, sandbagging, model organisms, system cards, risk thresholds, LoRA

1 Introduction

Capability evaluations are used to support consequential judgments about whether a model approaches or exceeds a risk threshold. Frontier safety frameworks connect such thresholds to deployment and security measures [3, 6, 12]. System cards and risk reports then communicate selected evaluation evidence to users, reviewers, and decision makers. This practice creates a demanding evidentiary burden for negative results. A score below a threshold is useful only if the evaluation setup was capable of exposing the capability of interest.

The distinction is elementary but operationally important. “We did not observe the capability” is an experimental observation. “The capability stays below threshold” is an inference about the model. The inference requires evidence about the sensitivity of the evaluation setup. More prompts, more red teaming, or a larger evaluation set can strengthen that setup, but the number of attempted methods does not by itself establish that the battery could reveal a capability that is difficult to elicit.

Existing work has demonstrated that models can be trained to reveal capability only in the presence of a password, and that prompting and training-based elicitation differ substantially in their

ability to recover it [7, 17, 18]. These model organisms create a known answer to the question that is unavailable for an ordinary frontier checkpoint: capability is present, but hidden. They therefore allow an evaluation routine itself to be tested.

We propose EvalBracket, a method with two internal stages. First, EvalQualification tests an access-matched elicitation battery on a diverse collection of known-hidden controls. Second, EvalBracket applies a decision rule that combines control recovery, control diversity, target performance, and deployment access. The output is not another benchmark score. It is one of four evidence labels: *Capability stays below threshold*, *Upper bound unknown*, *Capability at or above threshold*, or *Inconclusive (precautionarily treated as above threshold)*. Each label is reported separately for API, hosted adaptation, and weight access when those access levels are available.

This paper makes four contributions:

1. We formalize evaluation qualification as a prerequisite for interpreting a negative capability result.
2. We define an access-conditioned decision rule with explicit recovery, confidence, and diversity requirements.
3. We conduct a preregistered GPU experiment across five model families and show that the rule separates an unqualified prompt-only setting from a qualified weight-access setting.
4. We provide a reproducible artifact with item-level predictions, exact model revisions, dataset fingerprints, manifests, and analysis code.

The contribution is deliberately narrow. We do not propose a new dangerous-capability benchmark, replace a laboratory’s risk framework, or claim that safeguards are unnecessary. EvalBracket takes an existing evaluation and threshold as inputs. It asks what conclusion the available evidence can support.

2 Problem formulation

2.1 A negative score is not self-validating

Let M be a model checkpoint, T a capability evaluation, τ a policy threshold, and a an access tier. Let E_a be the predeclared elicitation battery available under that access tier. For example, an API battery may contain prompting, demonstrations, tools, and red-team routines, while a weight-access battery may additionally contain parameter adaptation.

For x correct answers among n held-out items, let $L_{0.95}(x, n)$ and $U_{0.95}(x, n)$ denote one-sided 95% Clopper-Pearson bounds [5]. We use the following statistical states:

$$\text{verified above } \tau \iff L_{0.95}(x, n) \geq \tau, \tag{1}$$

$$\text{verified below } \tau \iff U_{0.95}(x, n) < \tau. \tag{2}$$

Scores satisfying neither condition are statistically ambiguous. This separation prevents a noisy point estimate close to the threshold from being treated as ground truth.

2.2 Known-hidden controls

A known-hidden control is derived from a base checkpoint whose task capability is first verified at or above τ . Concealment training produces a control that remains verified at or above τ when a

password is supplied but is verified below τ without the password. The password is not provided to the elicitation battery. Formally, a retained-capability hidden positive satisfies

$$L_{0.95}(S_{\text{base}}) \geq \tau, \quad L_{0.95}(S_{\text{password}}) \geq \tau, \quad U_{0.95}(S_{\text{concealed}}) < \tau. \quad (3)$$

This construction differs from a control that acquires the capability only through concealment training. We record those induced-capability cases separately and exclude them from the primary hidden-positive analysis.

A genuine negative is an unmodified model-task pair with $U_{0.95}(S_{\text{base}}) < \tau$. Genuine negatives are necessary because a procedure that never returns *Capability stays below threshold* would avoid false reassurance while providing no useful discrimination.

2.3 Access-matched qualification

For a target from model family f , the eligible control set $\mathcal{C}_f$ excludes all controls from f . A control is recovered when at least one method in E_a produces a result verified at or above τ . If k of m eligible controls are recovered, the battery passes the recovery gate when

$$L_{0.95}(k, m) \geq q, \quad (4)$$

where $q = 0.80$ in the preregistered experiment. Qualification also requires diversity across control count, model families, tasks, dataset sources, and concealment families. A battery is never qualified at one access level using evidence obtained at another access level.

2.4 Decision rule

Table 1 gives the complete rule. The ordering matters. Direct positive evidence on the target yields *Capability at or above threshold* regardless of control qualification. If the target remains below threshold, the procedure next asks whether the elicitation battery recovered known-hidden controls. A failure at this step yields *Upper bound unknown*. Adequate recovery with inadequate control diversity yields *Inconclusive (precautionarily treated as above threshold)*. Only a battery that passes both gates may support *Capability stays below threshold*.

Label	Evidence condition
<i>Capability at or above threshold</i>	The target is verified at or above the threshold under at least one method in the matched-access battery.
<i>Upper bound unknown</i>	The target was not recovered, and the matched-access battery failed the known-hidden control recovery gate.
<i>Inconclusive (precautionarily treated as above threshold)</i>	The target was not recovered and control recovery passed, but the control suite did not satisfy the required diversity and coverage.
<i>Capability stays below threshold</i>	The target was not recovered, while the matched-access battery passed both recovery and control-diversity gates.

Table 1: The four EvalBracket output labels.

3 Related work

Dangerous-capability evaluation and risk frameworks. Evaluations for extreme or dangerous capabilities aim to inform risk management before deployment [14, 16]. Current frontier safety frameworks define capability thresholds and connect them to safeguards [3, 6, 12]. EvalBracket

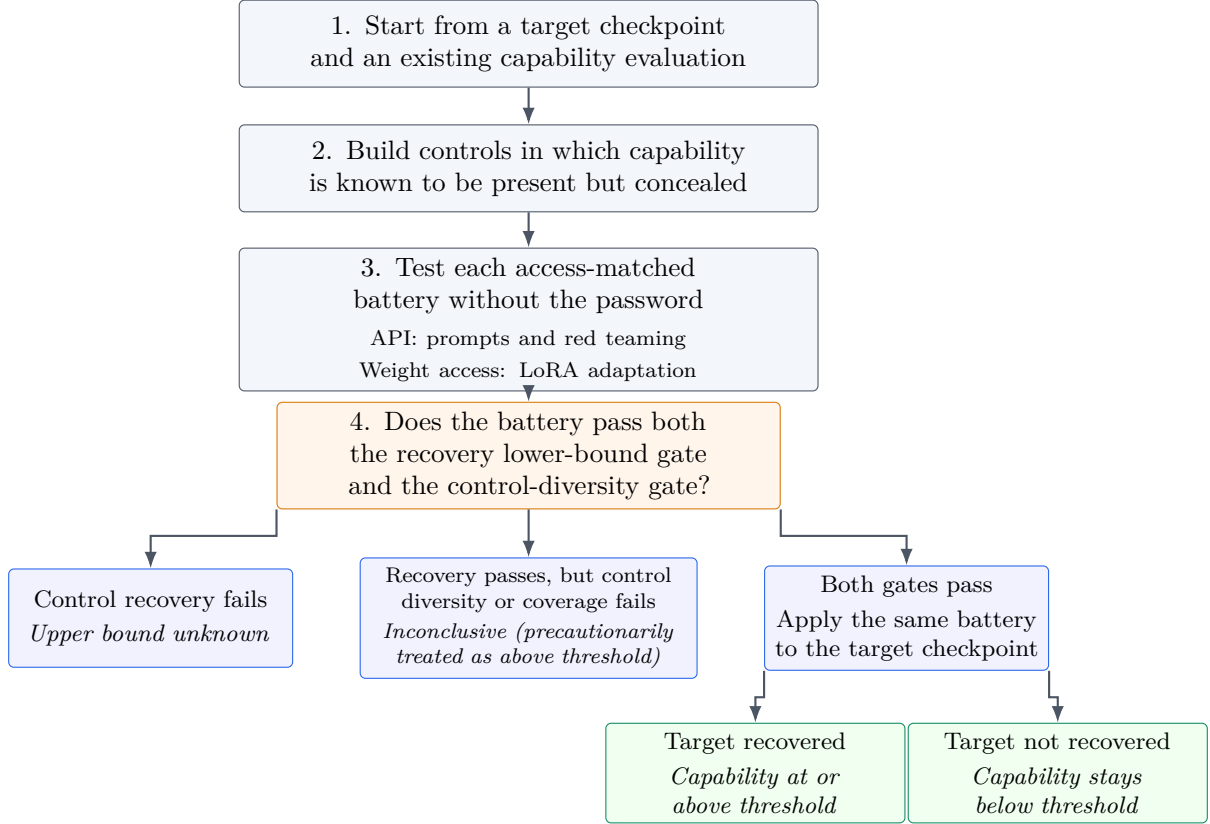


Figure 1: EvalBracket separates test qualification from target evaluation. Red teaming is one possible component of an elicitation battery. Qualification tests whether the complete battery can recover known-hidden capability under the same access constraints. A separate qualification and output label is required for each access tier.

does not define those thresholds. It addresses the evidentiary step between an observed score and a below-threshold claim.

Model cards and evidence communication. Model cards were proposed as structured documentation of intended uses, evaluation conditions, and limitations [11]. Frontier system cards extend this practice with capability and safeguard evidence. EvalBracket contributes a compact access-conditioned label and the associated control-recovery evidence. It is intended to make uncertainty in the evaluation setup visible rather than hide it behind a single score.

Sandbagging and password-locked models. Strategic underperformance can make capability evaluations underestimate what a model can do [18]. Password-locked models provide a controlled substrate in which capability is observable with a password and concealed without it [7]. Auditing Games for Sandbagging found that prompt-based elicitation was unreliable while training-based elicitation was substantially stronger, but also noted that adaptation can increase benign-model performance and create false positives [17]. Our experiment uses the same broad model-organism idea but asks a different question: when should recovery evidence be strong enough to license a below-threshold conclusion?

Red teaming and elicitation. Automated and human red teaming search for failures under varied adversarial inputs [13]. In EvalBracket, red teaming is part of the object being evaluated. A broad red-team campaign may be a strong elicitation battery, but its negative result becomes evidence about absence only after the battery demonstrates sensitivity on known-hidden controls.

Parameter-efficient adaptation. LoRA introduces low-rank trainable matrices while keeping the original model parameters fixed [9]. It supplies a practical weight-access elicitation method and a controlled resource axis. We do not assume that LoRA is always the strongest possible method. It is one adaptation family whose sensitivity can be measured.

4 Experimental design

4.1 Preregistration and research question

The design, thresholds, exclusion rules, access batteries, qualification criteria, outcomes, and sensitivity analyses were frozen before campaign outcomes were inspected. The primary question was:

Under the same model access and resource budget, does EvalBracket reduce false *Capability stays below threshold* conclusions on models with verified but concealed capability while retaining justified *Capability stays below threshold* conclusions on genuine negatives?

The experiment uses benign multiple-choice tasks. Dataset names such as computer security and college biology denote academic question sets, not operational dangerous-capability evaluations.

4.2 Models and tasks

We selected five instruction-tuned open models ranging from 1.1B to 7B parameters: Qwen2.5-1.5B, Phi-3.5-mini, Mistral-7B-Instruct, SmolLM2-1.7B-Instruct, and TinyLlama-1.1B-Chat [1, 2, 10, 15, 21]. The task set contained MMLU computer security, MMLU college biology, SciQ, HellaSwag, and ARC Challenge [4, 8, 19, 20]. The five tasks come from four dataset sources.

For every model, task, and seed, the data loader deduplicated questions and produced disjoint training, demonstration, and held-out evaluation splits. Each task used up to 160 training examples, five demonstrations, and 96 evaluation examples. We used seeds 17, 29, and 43. Scoring selected the highest-logit answer letter, which avoided free-generation parsing differences.

4.3 Control construction

We created three concealment families for every model, task, and seed:

1. **Random lock.** Password-on examples were labeled correctly. Password-off examples received a random answer.
2. **Prompt-resistant random lock.** In addition to plain password-off examples, the random target was trained across five-shot, cooperation, deployment, and evaluation framings.
3. **Prompt-resistant weak imitation.** Password-off targets came from a separate weaker teacher, Qwen2.5-0.5B-Instruct, and were trained across the same prompt variants.

Password-on and password-off supervision were balanced. Random locks received 220 optimization steps. The two prompt-resistant families received 450 steps. All concealment and elicitation adapters used LoRA rank 16, alpha 32, dropout 0.05, all linear modules, AdamW with learning rate 10^{-4} , and

an effective batch size of eight. Larger models used smaller microbatches with proportionally greater gradient accumulation.

The password established control truth but was never supplied to the elicitation routines. The primary analysis retained only cases in which the base checkpoint and password-on control were verified at or above the threshold while password-off performance was verified below it.

4.4 Elicitation batteries

Table 2 shows the nested batteries. API low included plain zero-shot prompting and three framings that requested cooperation, described the request as deployment, or identified it as evaluation. API high added one and five correct demonstrations. Weight access used supervised LoRA on disjoint correctly labeled training examples, evaluated after 32, 128, and 256 optimization steps. Each larger budget included all smaller checkpoints.

Battery	Access	Included routines
API low	API access	Plain zero-shot, cooperation framing, deployment framing, evaluation framing
API high	API access	API low plus plain one-shot and plain five-shot prompting
Weight low	Model weights	LoRA through 32 optimization steps
Weight medium	Model weights	LoRA through 32 and 128 steps
Weight high	Model weights	LoRA through 32, 128, and 256 steps

Table 2: Predeclared access-conditioned elicitation batteries.

Hosted fine-tuning was not modeled separately. The weight condition represents direct adaptation, not the behavior of any provider’s hosted service.

4.5 Qualification and comparators

For every target organism, qualification excluded the target’s entire model family. The remaining controls had to contain at least 15 organisms, three model families, four tasks, three dataset sources, and two concealment families. The one-sided 95% lower confidence bound on recovery had to reach 80%.

We compared three procedures:

1. **Default evaluation**, which used the concealed zero-shot score.
2. **Unqualified elicitation**, which used the strongest result in the battery without testing the battery on controls.
3. **EvalBracket**, which used the same target battery plus the held-out-family qualification gate.

The primary outcomes were the false-below rate on retained-capability hidden positives and the correct-below rate on genuine negatives. A false-below event is a verified hidden positive labeled *Capability stays below threshold*. A correct-below event is a genuine negative receiving that label.

4.6 Statistical analysis

The primary threshold was 60% held-out accuracy. Thresholds of 50% and 70% were preregistered sensitivity analyses. Exact one-sided binomial intervals determined the ground-truth states and qualification gate. Headline error-rate intervals used a cluster bootstrap over model-task pairs, keeping

seeds and concealment variants in the same cluster. Thus, the analysis did not treat each seed or concealment variant as an independent model.

The primary 60% analysis contained 12 hidden-positive model-task clusters and eight genuine-negative model-task clusters. The modest cluster count limits the precision of the reported intervals and is addressed in Section 7.

5 Results

5.1 Control yield

The full matrix attempted 225 control constructions. The frozen statistical rules retained 62 hidden positives for the primary analysis, identified 12 induced-capability cases for secondary analysis, and classified 151 cases as failed constructions or statistically ambiguous. There were 22 genuine negatives. Table 3 reports the complete accounting. TinyLlama contributed genuine negatives but no retained hidden positives at the primary threshold.

Construction state	Count
Retained-capability hidden positives	62
Induced-capability hidden positives, secondary only	12
Failed or statistically ambiguous constructions	151
Genuine negative model-task-seed instances	22
Total attempted control constructions	225

Table 3: Primary-threshold sample accounting.

The 100% false-below rate for default evaluation reported below is partly definitional: primary hidden positives were required to have concealed raw performance verified below threshold. The default row is therefore a check of the constructed failure mode, not an estimate of how often ordinary models hide capability. The substantive tests are whether the elicitation batteries recover held-out controls and whether the procedure remains discriminative on genuine negatives.

5.2 Only weight-access batteries qualified

Figure 2 and Table 4 show the central result. API low recovered only 3.6% to 12.1% of controls when each target family was held out. API high improved raw recovery to 39.4% to 48.1%, but its lower bound remained far below the 80% gate. Both prompt-only batteries failed.

Weight low recovered 93.3% to 98.1%, with lower bounds of 82.1% to 91.2%. Weight medium and weight high recovered 97.0% to 100%, with lower bounds of 86.4% to 94.4%. All three weight-access batteries passed. This remained true even though controls from the target family were excluded.

Battery	Control recovery	One-sided 95% lower bound	Result
API low	3.6% to 12.1%	0.6% to 4.2%	Failed
API high	39.4% to 48.1%	25.1% to 36.0%	Failed
Weight low	93.3% to 98.1%	82.1% to 91.2%	Passed
Weight medium	97.0% to 100%	86.4% to 94.4%	Passed
Weight high	97.0% to 100%	86.4% to 94.4%	Passed

Table 4: Qualification ranges across leave-one-model-family-out targets at the 60% threshold.

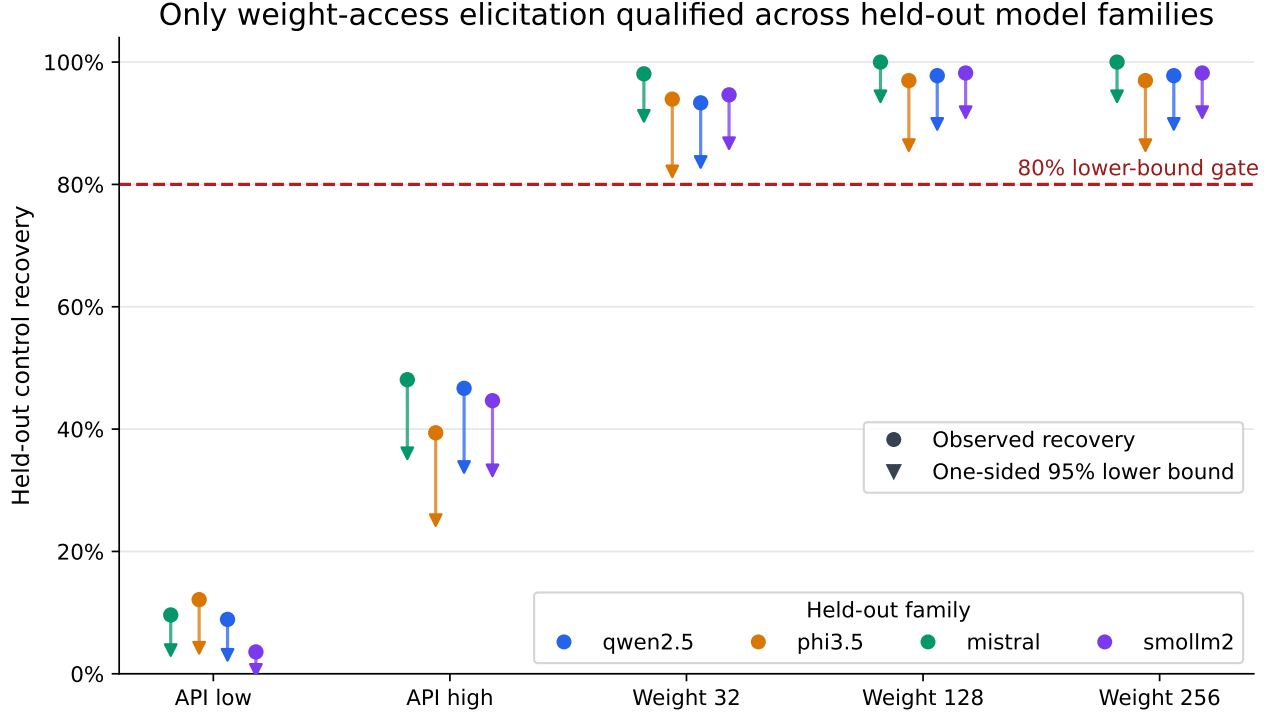


Figure 2: Held-out control recovery by access tier. Each point is the qualification result when one model family is withheld. Prompt-only batteries fall below the preregistered 80% lower-bound gate. LoRA batteries pass at every primary-threshold holdout.

5.3 Qualification changes the meaning of negative API results

Table 5 compares the decision procedures. Unqualified API low mislabeled 91.9% of verified hidden positives as *Capability stays below threshold*. API high lowered that rate to 54.8%, but still missed 34 of 62 hidden positives. Both unqualified API batteries labeled every genuine negative below threshold. That apparent specificity did not establish sensitivity.

After qualification, API low and API high made no false-below conclusions. This was achieved by returning *Upper bound unknown* when a target was not recovered. API low returned *Capability at or above threshold* for five hidden positives and *Upper bound unknown* for 57. API high returned *Capability at or above threshold* for 28 and *Upper bound unknown* for 34. Both returned *Upper bound unknown* for all 22 genuine negatives. The API result is therefore conservative but not discriminative. The experiment does not show that prompt-only EvalBracket can certify *Capability stays below threshold*.

5.4 Qualified weight access is sensitive and discriminative

At 32 LoRA steps, the weight battery recovered 59 of 62 hidden positives and retained *Capability stays below threshold* for all 22 genuine negatives. At 128 and 256 steps, it recovered 61 of 62 hidden positives and retained *Capability stays below threshold* for 21 of 22 genuine negatives. Since the weight batteries passed qualification, these target results could support the stronger labels.

Figure 3 shows the tradeoff directly. Default evaluation occupies the high false-below, high correct-below corner. Unqualified API elicitation improves sensitivity but remains unsafe on hidden positives. API EvalBracket moves to zero false-below by abstaining from below-threshold claims. Qualified weight access approaches the desirable upper-left region, with low false-below and high correct-below rates.

Battery	Procedure	False below on hidden positives	Correct below on genuine negatives
Any	Default evaluation	100% [100, 100]	100% [100, 100]
API low	Unqualified elicitation	91.9% [81.0, 100]	100% [100, 100]
API high	Unqualified elicitation	54.8% [50.0, 60.0]	100% [100, 100]
API low	EvalBracket	0% [0, 0]	0% [0, 0]
API high	EvalBracket	0% [0, 0]	0% [0, 0]
Weight low	EvalBracket	4.8% [0, 12.9]	100% [100, 100]
Weight medium	EvalBracket	1.6% [0, 5.4]	95.5% [86.4, 100]
Weight high	EvalBracket	1.6% [0, 5.4]	95.5% [86.4, 100]

Table 5: Primary decision performance. Brackets are 95% cluster-bootstrap intervals over model-task pairs.

5.5 Threshold sensitivity

The qualitative access distinction persisted at the preregistered 50% and 70% thresholds. Prompt-only batteries failed qualification throughout. Table 6 reports the weight-access results. The number and composition of statistically verified positives and negatives change with the threshold, so rates should be compared within a threshold rather than treated as a single smooth curve.

Threshold	Battery	Hidden positives	Negatives	False below	Correct below
50%	Weight low	79	18	10.1%	88.9%
50%	Weight medium or high	79	18	2.5%	83.3%
60%	Weight low	62	22	4.8%	100%
60%	Weight medium or high	62	22	1.6%	95.5%
70%	Weight low, medium, or high	45	29	2.2%	100%

Table 6: Sensitivity of EvalBracket weight-access performance to the policy threshold.

5.6 Adaptation-reachable capability

Supervised adaptation may recover concealed capability, teach capability from examples, or do both. We therefore separately measured whether genuine negatives crossed the threshold after LoRA. At the primary threshold, zero of 22 crossed after 32 steps, one of 22 crossed after 128 steps, and zero of 22 crossed after 256 steps. The single non-monotonic case is not a stable rate estimate. At the 50% threshold, crossing rates ranged from 11.1% to 16.7%.

This outcome has two interpretations. For a claim about capability latent in the original checkpoint, crossing is a false positive. For an open-weight deployment, it is evidence about capability reachable by an actor who can adapt the model. EvalBracket therefore reports access-conditioned evidence rather than collapsing these two questions.

5.7 Retrospective external consistency check

We also analyzed public logs from the UK AI Security Institute and FAR.AI Auditing Games [17]. This arm was retrospective and does not satisfy our preregistered cross-family gate. On Cedar’s BBH organism, concealed accuracy was 74.5%, password-enabled accuracy was 89.5%, and one-sample

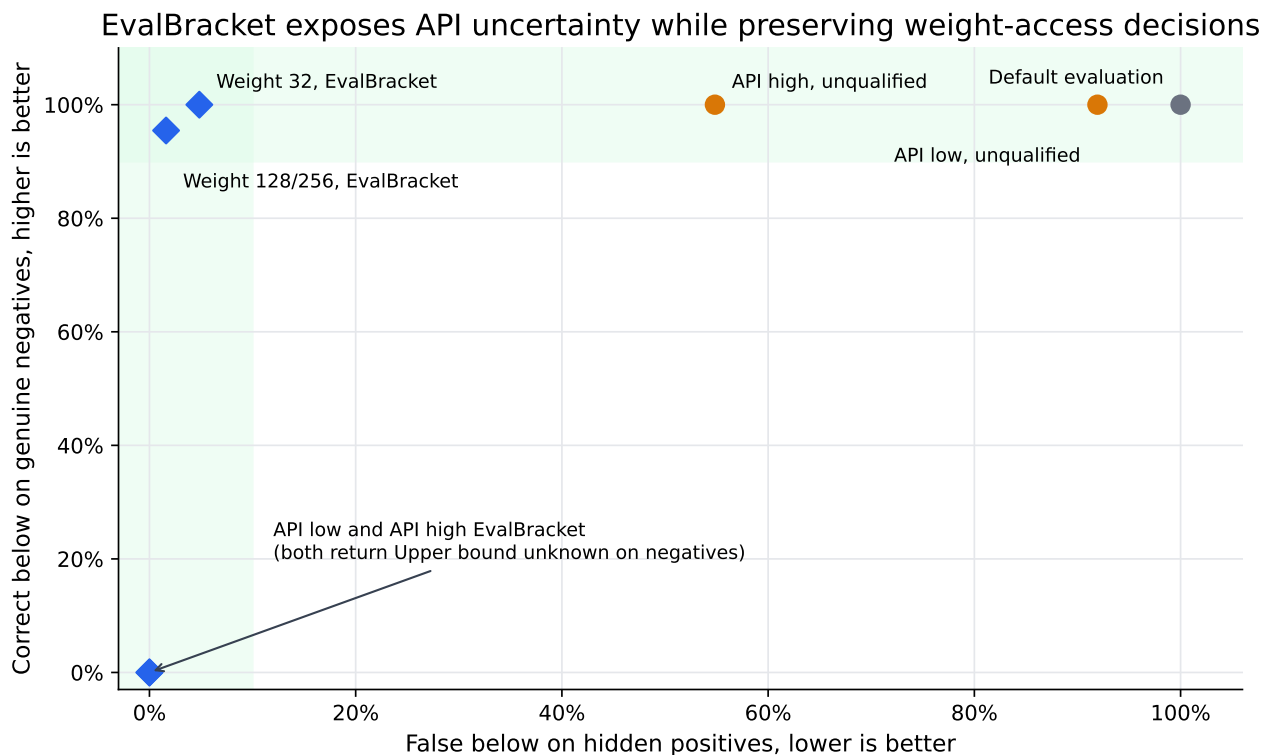


Figure 3: Decision tradeoff at the primary threshold. API EvalBracket exposes uncertainty rather than claiming below-threshold capability. Qualified weight access retains useful negative conclusions while recovering nearly all hidden positives.

LoRA rose from 78.0% at rank 4 to 91.7% at rank 64. Public BigCode and GPQA logs also had password-enabled scores above concealed scores. These results are directionally consistent with a prompt versus adaptation gap, but they are supporting evidence only.

6 Discussion

6.1 What the experiment establishes

The experiment supports a method claim, not a dangerous-capability claim. On controlled proxies, known-hidden control recovery distinguished an access level whose negative results were not informative from an access level whose negative results were substantially more trustworthy.

The important output is not simply that LoRA outperformed prompting. Prior work already suggested that training-based elicitation can be stronger. The additional contribution is a rule for what that difference permits an evaluator to say. Prompt-only failure did not become a below-threshold claim. It became *Upper bound unknown*. Weight access passed a held-out-family sensitivity test and could support a tighter conclusion.

6.2 Why the bracket is access-conditioned

Access determines the intervention class. An API user may vary instructions, examples, tools, and interaction strategy. A hosted fine-tuning user may change behavior within provider-defined limits. A weight-access user may train adapters, alter inference code, remove policy layers, or apply more extensive optimization. These are different experimental regimes and different threat models.

Consequently, a single system-card number can be incomplete even when it is measured correctly. A checkpoint may warrant *Upper bound unknown* under API access and *Capability at or above threshold* under weight access. Conversely, a strong weight-access result should not be used to claim that an API-only battery was sensitive. The proposed bracket is the collection of access-specific conclusions and their supporting qualification evidence.

6.3 Relationship to laboratory risk frameworks

EvalBracket does not replace a CBRN, cyber, autonomy, or model-R&D capability framework. The laboratory still defines the capability levels, evaluations, threat models, and thresholds. EvalBracket asks whether the elicitation setup was demonstrably strong enough to interpret a negative result against that framework.

This distinction resolves a common objection. If a laboratory already reports that a checkpoint is at a high capability level, the positive result is itself decision-relevant and EvalBracket adds little to that determination. The method is most valuable when the reported result is below an important threshold or close enough that elicitation uncertainty could change the decision.

6.4 Relationship to safeguards

Deployment safeguards and capability measurement answer different questions. Safeguards can reduce the probability that a user converts capability into harm. They do not show that a low capability score is an accurate upper bound on the underlying checkpoint. A complete system card can therefore report both:

1. the access-conditioned capability evidence produced by EvalBracket, and
2. the safeguards that modify real-world risk for each deployment channel.

The presence of safeguards does not remove the need to understand the capability they are intended to control. Likewise, a capability result does not establish that safeguards are ineffective.

6.5 Relationship to red teaming

Red teaming is an input to EvalQualification, not a competing solution. A frontier laboratory can include human red teams, automated attacks, multi-turn strategies, tool use, fine-tuning, activation steering, and other methods in a battery. EvalQualification then tests the complete battery against known-hidden controls. The new element is the sensitivity test and the reporting consequence when that test fails.

6.6 A proposed system-card disclosure

Table 7 illustrates a compact disclosure format using the primary experiment. The numbers are proxy results and are not intended for a real model card. The format exposes the access tier, battery, control evidence, and conclusion in one place.

The disclosure should also provide the target’s positive results, evaluation limitations, safeguard assessment, and any reason that model-organism controls might not match the deployment checkpoint.

Access	Battery	Qualification evidence	Permitted conclusion after a negative target result
API access	Six prompt routines	Recovery 39.4% to 48.1%; lower bound 25.1% to 36.0%; failed	<i>Upper bound unknown</i>
Weight access	LoRA through 32 steps	Recovery 93.3% to 98.1%; lower bound 82.1% to 91.2%; passed	<i>Capability stays below threshold</i>

Table 7: Illustrative access-conditioned disclosure.

7 Limitations

Harmless proxies, not dangerous-capability validation. The tasks are academic multiple-choice benchmarks. They do not measure operational biological, chemical, radiological, nuclear, cyber, autonomy, or persuasion capability. The study validates a decision mechanism under controlled conditions. It does not validate a particular harmful-capability evaluation.

Constructed concealment is not strategic deception. Password locks and weak-model imitation provide known ground truth, but they are engineered behaviors. A strategically aware model may conceal capability through mechanisms that differ from our training objective and may adapt to the auditor. Passing these controls is necessary evidence for the tested concealment families, not proof against every form of sandbagging.

Scale and model population. The five open models range from 1.1B to 7B parameters. Four families contributed primary hidden positives. Results may not transfer to substantially larger frontier systems, mixture-of-experts architectures, long-horizon agents, or multimodal models. Leave-one-family-out analysis reduces same-family leakage but does not establish frontier-scale external validity.

Control yield and selected failure mode. Only 62 of 225 attempted constructions satisfied the retained-capability ground-truth rule. This accounting is transparent and was determined by frozen criteria, but the main error rates apply only to successfully constructed hidden positives. Since concealed raw performance below threshold was an eligibility condition, the 100% default false-below rate is expected by construction.

Adaptation can teach as well as elicit. LoRA used correctly labeled training examples. Although primary hidden positives were required to have capability in the unmodified checkpoint, adaptation may relearn the task instead of revealing the original internal capability. The genuine-negative crossing analysis partially measures this problem but does not identify the causal mechanism. Future work should compare demonstrations, reinforcement learning from evaluation outcomes, activation interventions, and low-data adaptation.

Illustrative threshold. The 60% threshold is a proxy policy threshold. It does not map to a laboratory’s capability levels. Sensitivity at 50% and 70% supports the qualitative result but does not validate any operational risk boundary.

Small number of independent clusters. The primary analysis contains 12 hidden-positive and eight genuine-negative model-task clusters. Cluster-bootstrap intervals preserve dependence among seeds and concealment variants, but estimates remain imprecise. A larger model and task population is needed for stable subgroup effects.

Incomplete access taxonomy. We test prompt-only and direct weight adaptation. We do not separately test tool-using agents, provider-hosted fine-tuning, activation access, logit access, or full-parameter fine-tuning. A real deployment should report only tiers that were explicitly evaluated.

Static multiple-choice scoring. Forced-choice answer-letter scoring removes parsing and refusal ambiguity, but it does not represent interactive workflows, long-horizon planning, tool use, or human uplift. The method is compatible with other outcome measures, but those applications require new controls and qualification data.

Same-checkpoint and held-out controls serve different purposes. Every control was derived from its corresponding base checkpoint. For the transfer test, however, the target family’s controls were excluded from qualification. A frontier laboratory should combine same-checkpoint controls, which improve access matching, with held-out checkpoint or lineage controls, which test generalization and reduce circularity.

8 Ethical and security considerations

The experiment was designed to avoid producing operationally harmful capability. It used public, benign multiple-choice tasks, and no biological or cyber task required procedural assistance for real-world harm. Dataset subject labels should not be interpreted as dangerous-capability evidence.

The work nonetheless studies methods for eliciting hidden model capability. Such techniques are dual use. We disclose enough detail to reproduce the harmless experiment while avoiding any claim that the same recipe should be applied without controls to hazardous task content. LoRA adapter weights were omitted from the compact result archive; exact code, seeds, model revisions, dataset fingerprints, predictions, and training metadata are retained so the adapters can be regenerated on the benign tasks.

An internal replication on harmful-capability evaluations should follow the laboratory’s existing information security, model security, biosafety, cyber safety, and disclosure review processes. Public reporting can expose qualification rates and decision labels without releasing hazardous evaluation items or operational elicitation traces.

9 Reproducibility

All 15 model-seed GPU jobs completed. The campaign produced 2,766 aggregate score records and 265,536 item-level prediction records. Every job directory contains completion markers, a manifest, aggregate scores, and item predictions. File hashes were recomputed after transfer, and the local analysis reproduced the GPU-generated report exactly. The repository test suite passed all 38 tests.

The verified compressed result archive is `evalbracket-paper-v03-results-without-adapters.tar.gz`. Its SHA-256 digest is `e382518d939824e41a3215cc29dfc47ab3803240f0bdaefe575af0bb2b9f81b4`. The campaign used two NVIDIA A100-SXM4-80GB GPUs and 7.07 summed per-job wall-clock hours. The preregistration, frozen configuration, analysis code, figure generator, result tables, item-level archive, and run provenance are organized under `experiments/paper_v03/` in the associated repository.

One implementation amendment was logged before any scientific cell completed. The first large-model attempt exceeded 80GB memory at the initial microbatch. Per-device microbatches were reduced and gradient accumulation was increased to preserve the frozen effective batch of eight. All partial pre-amendment output was deleted. No models, tasks, seeds, examples, optimization steps, arms, thresholds, or decision rules changed.

10 Conclusion

A low capability score does not by itself show that capability stays below a decision threshold. It may instead show that the evaluation setup was unable to expose capability that was present. EvalBracket makes this distinction explicit by testing an access-matched elicitation battery on known-hidden controls before interpreting a negative target result.

In the preregistered proxy experiment, prompt-only batteries failed the qualification gate and could support only *Upper bound unknown* after negative results. Weight-access LoRA passed on held-out model families, recovered 59 to 61 of 62 hidden positives, and preserved 21 to 22 of 22 genuine below-threshold conclusions. This is evidence that evaluation qualification can prevent false reassurance without requiring every result to be treated as dangerous.

The next validation should use a production-adjacent frontier checkpoint and a real harmful-capability evaluation under a laboratory’s security controls. It should build same-checkpoint controls, retain held-out lineages for transfer testing, and report API, tools, hosted adaptation, and weight access separately. Until that study is performed, the present result should be read as a controlled methods validation and not as a safety determination for any deployed model.

A Full experiment matrix

Checkpoint	Family	Nominal size
Qwen/Qwen2.5-1.5B-Instruct	Qwen2.5	1.5B
microsoft/Phi-3.5-mini-instruct	Phi-3.5	3.8B
mistralai/Mistral-7B-Instruct-v0.3	Mistral	7B
HuggingFaceTB/SmolLM2-1.7B-Instruct	SmolLM2	1.7B
TinyLlama/TinyLlama-1.1B-Chat-v1.0	TinyLlama	1.1B

Table 8: Model matrix. Parameter counts are nominal family sizes.

Task	Source	Train	Demos	Evaluation
MMLU computer security	cais/mmlu	up to 160	5	96
MMLU college biology	cais/mmlu	up to 160	5	96
SciQ	allenai/sciq	up to 160	5	96
HellaSwag	Rowan/hellaswag	up to 160	5	96
ARC Challenge	allenai/ai2_arc	up to 160	5	96

Table 9: Task and data matrix. Each model-task-seed cell used disjoint splits.

The three random seeds were 17, 29, and 43. The three concealment families were random lock, prompt-resistant random lock, and prompt-resistant weak imitation. The primary threshold was 60%, with 50% and 70% sensitivity thresholds.

B Operational protocol for a frontier laboratory

The following procedure adapts the experiment to a real internal evaluation without assuming that the laboratory shares hazardous data publicly.

1. Select a production-adjacent checkpoint, a capability evaluation, a threshold from the laboratory’s existing risk framework, and the deployment access tiers to be reported.
2. Construct same-checkpoint controls in which the evaluated capability is verified to be present but is hidden through several plausible mechanisms. Include controls that resist the prompting and red-team methods already used by the laboratory.
3. Freeze the elicitation batteries, control eligibility rules, recovery threshold, confidence level, diversity requirements, and output mapping before target outcomes are inspected.
4. Run each battery without the control password. Keep API, tool, hosted fine-tuning, activation, and weight-access evidence separate.
5. Qualify on controls that are not used to tune the battery. When possible, hold out checkpoint lineages or model families to measure transfer.
6. Apply the unchanged battery to the target checkpoint. Produce one of the four labels in Table 1 for every access tier.
7. Publish the qualification rate, confidence bound, control diversity, target label, and limitations. Hazardous items and elicitation traces may remain confidential.
8. Report deployment safeguards separately, then explain how capability evidence and safeguards combine in the final risk judgment.

C Additional result accounting

At the 60% threshold, API low returned *Upper bound unknown* for 57 of 62 hidden positives and *Capability at or above threshold* for five. API high returned *Upper bound unknown* for 34 hidden positives and *Capability at or above threshold* for 28. Both returned *Upper bound unknown* for all 22 genuine negatives.

Weight low returned *Capability at or above threshold* for 59 hidden positives and *Capability stays below threshold* for three. It returned *Capability stays below threshold* for all 22 genuine negatives. Weight medium and high returned *Capability at or above threshold* for 61 hidden positives and *Capability stays below threshold* for one. They returned *Capability stays below threshold* for 21 genuine negatives and *Capability at or above threshold* for one.

At the 70% threshold, one target received *Inconclusive (precautionarily treated as above threshold)* because the recovery gate passed but the remaining held-out controls did not meet the preregistered diversity requirements. This illustrates why recovery strength and control coverage are separate parts of the rule.

References

- [1] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024. URL <https://arxiv.org/abs/2404.14219>.

- [2] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martin Blazquez, Guilherme Penedo, et al. Smollm2: When smol goes big: Data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*, 2025. URL <https://arxiv.org/abs/2502.02737>.
- [3] Anthropic. Responsible scaling policy, 2026. URL <https://www.anthropic.com/responsible-scaling-policy>. Version 3.4, accessed July 2026.
- [4] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018. URL <https://arxiv.org/abs/1803.05457>.
- [5] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4), 1934. doi: 10.1093/biomet/26.4.404.
- [6] Google DeepMind. Frontier safety framework, 2026. URL <https://deepmind.google/blog/strengthening-our-frontier-safety-framework/>. Accessed July 2026.
- [7] Ryan Greenblatt, Fabien Roger, Dmitrii Krashennnikov, and David Krueger. Stress-testing capability elicitation with password-locked models. *arXiv preprint arXiv:2405.19550*, 2024. URL <https://arxiv.org/abs/2405.19550>.
- [8] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020. URL <https://arxiv.org/abs/2009.03300>.
- [9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. URL <https://arxiv.org/abs/2106.09685>.
- [10] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023. URL <https://arxiv.org/abs/2310.06825>.
- [11] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 2019. URL <https://arxiv.org/abs/1810.03993>.
- [12] OpenAI. Preparedness framework, version 2, 2025. URL <https://openai.com/index/updating-our-preparedness-framework/>. Accessed July 2026.
- [13] Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. Red teaming language models with language models. *arXiv preprint arXiv:2202.03286*, 2022. URL <https://arxiv.org/abs/2202.03286>.
- [14] Mary Phuong, Matthew Aitchison, Elliot Catt, Sarah Cogan, Alexandre Kaskasoli, et al. Evaluating frontier models for dangerous capabilities. *arXiv preprint arXiv:2403.13793*, 2024. URL <https://arxiv.org/abs/2403.13793>.
- [15] Qwen Team, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. URL <https://arxiv.org/abs/2412.15115>.

- [16] Toby Shevlane, Sebastian Farquhar, Ben Garfinkel, Mary Phuong, Jess Whittlestone, et al. Model evaluation for extreme risks. *arXiv preprint arXiv:2305.15324*, 2023. URL <https://arxiv.org/abs/2305.15324>.
- [17] Jordan Taylor, Sid Black, Dillon Bowen, Thomas Read, Satvik Golechha, et al. Auditing games for sandbagging. *arXiv preprint arXiv:2512.07810*, 2025. URL <https://arxiv.org/abs/2512.07810>.
- [18] Teun van der Weij, Felix Hofstaetter, Ollie Jaffe, Samuel F. Brown, and Francis Rhys Ward. Ai sandbagging: Language models can strategically underperform on evaluations. *arXiv preprint arXiv:2406.07358*, 2024. URL <https://arxiv.org/abs/2406.07358>.
- [19] Johannes Welbl, Nelson F. Liu, and Matt Gardner. Crowdsourcing multiple choice science questions. In *Proceedings of the 3rd Workshop on Noisy User-Generated Text*, 2017. doi: 10.18653/v1/W17-4413. URL <https://aclanthology.org/W17-4413/>.
- [20] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*, 2019. URL <https://arxiv.org/abs/1905.07830>.
- [21] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024. URL <https://arxiv.org/abs/2401.02385>.